1.2.3 - Newton-Raphson: Cube Root

ADHARVAN KSHATHRIYA BOMMAGANI - EE25BTECH11003

March 25, 2026

Question

The Newton-Raphson iteration formula to find a cube root of a positive number c is:

a) $x_{k+1} = \dfrac{2x_k^3 + \sqrt[3]{c}}{3x_k^2}$

b) $x_{k+1} = \dfrac{2x_k^3 - \sqrt[3]{c}}{-3x_k^2}$

c) $x_{k+1} = \dfrac{2x_k^3 + c}{3x_k^2}$

d) $x_{k+1} = \dfrac{x_k^3 + c}{3x_k^2}$

Step 1: Define the Goal

We want to find $x = \sqrt[3]{c}$, i.e., $x^3 = c$, which means:

$$x^3 - c = 0$$

So we define:

$$f(x) = x^3 - c$$

Finding $\sqrt[3]{c}$ is the same as finding the **root** of $f(x) = x^3 - c$

A root is the value of x where $f(x) = 0$

Step 2: Newton-Raphson Formula

The general Newton-Raphson iteration formula is:

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

$x_k \rightarrow$ current guess at step k

$x_{k+1} \rightarrow$ next improved guess

$f(x_k) \rightarrow$ how far we are from zero

$f'(x_k) \rightarrow$ slope (derivative) at current guess

Step by Step Derivation

What we want:

Find x where $f(x) = 0$ (called the root)

STEP 1 — Start with a Guess x_k

Assume a function $f(x)$ and an initial guess x_k

At that point, function value is $f(x_k)$

Draw Tangent Line

The tangent line at point $(x_k, f(x_k))$ has:

Slope $= f'(x_k)$

Passes through $(x_k, f(x_k))$

Equation of tangent line:

$$y - f(x_k) = f'(x_k)(x - x_k)$$

Intersection with x-axis

At x-axis, $y = 0$

$$0 - f(x_k) = f'(x_k)(x - x_k)$$

$$-f(x_k) = f'(x_k)(x - x_k)$$

Divide both sides by $f'(x_k)$:

$$\frac{-f(x_k)}{f'(x_k)} = x - x_k$$

Final Rearrangement

Rearranging:

$$x = x_k - \frac{f(x_k)}{f'(x_k)}$$

This becomes the next approximation:

$$\boxed{x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}}$$

Step 3: Compute $f'(x)$

We have:

$$f(x) = x^3 - c$$

Differentiating term by term using the Power Rule $\frac{d}{dx}(x^n) = nx^{n-1}$:

$$f'(x) = \frac{d}{dx}(x^3) - \frac{d}{dx}(c) \tag{1}$$

$$= 3x^{3-1} - 0 \tag{2}$$

$$= 3x^2 \tag{3}$$

$\frac{d}{dx}(c) = 0$ because c is a constant

Step 4: Substitute into Formula

Substituting $f(x_k) = x_k^3 - c$ and $f'(x_k) = 3x_k^2$:

$$x_{k+1} = x_k - \frac{x_k^3 - c}{3x_k^2} \tag{4}$$

Combine over a common denominator:

$$x_{k+1} = \frac{3x_k^3 - (x_k^3 - c)}{3x_k^2} \tag{5}$$

$$= \frac{3x_k^3 - x_k^3 + c}{3x_k^2} \tag{6}$$

$$= \frac{2x_k^3 + c}{3x_k^2} \tag{7}$$

Step 5: Identify the Correct Option

Option	Formula	Correct?
a)	$\dfrac{2x_k^3 + \sqrt[3]{c}}{3x_k^2}$	$\times$ (has $\sqrt[3]{c}$ not c)
b)	$\dfrac{2x_k^3 - \sqrt[3]{c}}{-3x_k^2}$	$\times$ (wrong signs)
c)	$\dfrac{2x_k^3 + c}{3x_k^2}$	✓ **Correct!**
d)	$\dfrac{x_k^3 + c}{3x_k^2}$	$\times$ (has x_k^3 not $2x_k^3$)

Numerical Verification: Finding $\sqrt[3]{8}$

Goal: Find $\sqrt[3]{8}$ (true answer $= 2$), with $c = 8$, $x_0 = 1$

Formula: $x_{k+1} = \dfrac{2x_k^3 + 8}{3x_k^2}$

Step k	x_k	x_{k+1}	Error $\lvert x_k - 2 \rvert$
0	1.000	3.333	1.000
1	3.333	2.462	1.333
2	2.462	2.081	0.462
3	2.081	2.003	0.081
4	2.003	2.000	0.003
5	**2.000**	**2.000**	≈ 0

Convergence Behaviour

Each step uses $x_{k+1} = x_k - \dfrac{f(x_k)}{f'(x_k)}$

Geometrically: tangent line at x_k crosses x-axis at x_{k+1}

$f(x_k)$ tells how big the correction should be

$f'(x_k)$ tells which direction to correct

Error decreases **rapidly** each iteration

Newton-Raphson has **quadratic convergence**

After 5 iterations: answer accurate to 3 decimal places

Final Answer

$$\boxed{x_{k+1} = \frac{2x_k^3 + c}{3x_k^2}}$$

Correct option: **(c)**

Python Code: Newton-Raphson for Cube Root

```
import numpy as np
import matplotlib.pyplot as plt

def f(x, c):
    return x**3 - c

def f_prime(x):
    return 3 * x**2

def newton_raphson(c, x0, tol=1e-10, max_iter=100):
    x = x0
    history = [x]
    for _ in range(max_iter):
        x_new = x - f(x, c) / f_prime(x)
        history.append(x_new)
        if abs(x_new - x) < tol:
            break
        x = x_new
    return x_new, history
```

Python Code: Plot Convergence

```
c = 8
x0 = 1.0
root, history = newton_raphson(c, x0)

errors = [abs(h - 2.0) for h in history]

plt.figure(figsize=(8, 4))
plt.semilogy(errors, marker='o')
plt.xlabel("Iteration")
plt.ylabel("Error |x_k - 2|")
plt.title("Convergence of Newton-Raphson")
plt.grid(True)
plt.tight_layout()
plt.savefig("convergence.png", dpi=150)
plt.show()
```

Plot of the error

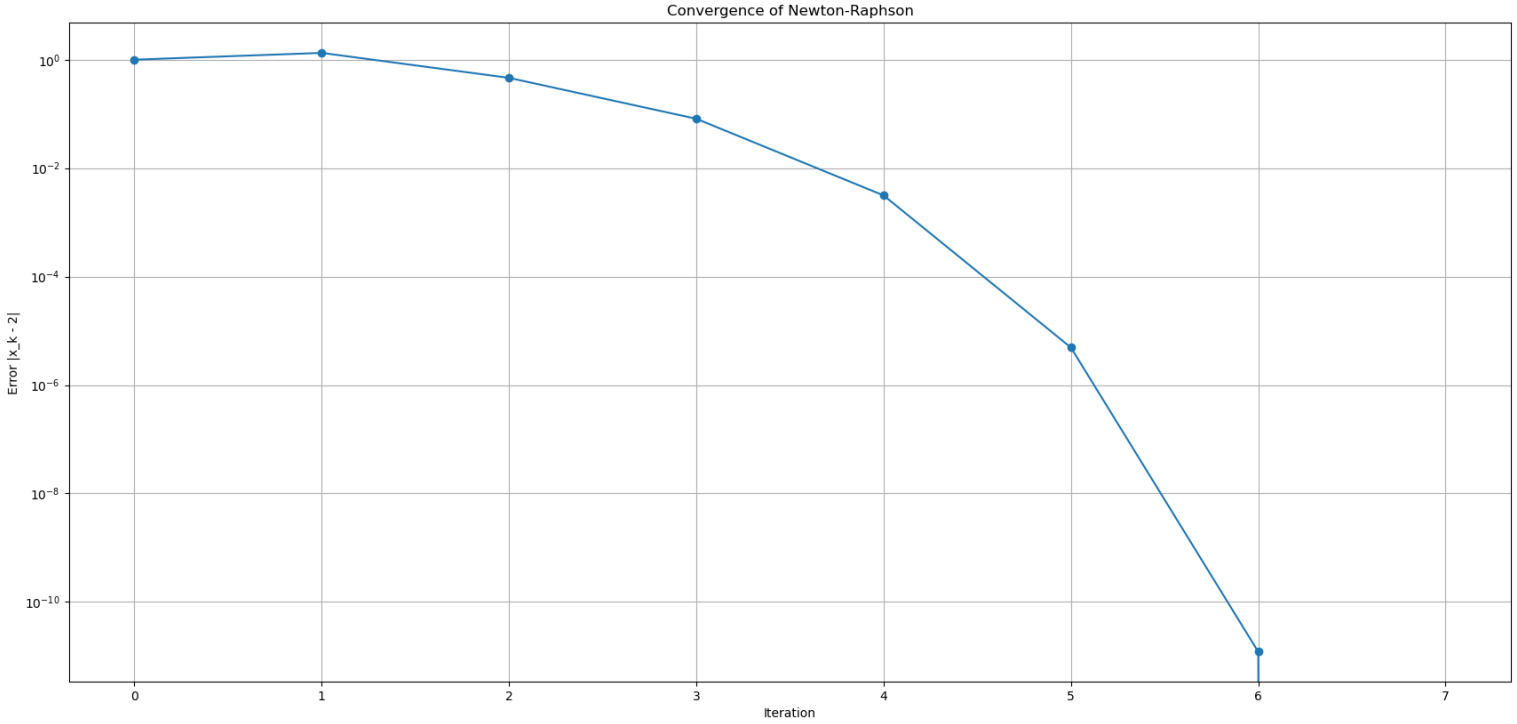


Figure: Plot of Error

Python Code: Newton-Raphson for Cube Root

```
import matplotlib.pyplot as plt
# Define the function and its derivative
def f(x):
    return x**3 - 8
def df(x):
    return 3*x**2
# Range for plotting the function curve
x_vals = np.linspace(1.5, 4.5, 400)
y_vals = f(x_vals)
plt.plot(x_vals, y_vals, label=r'$f(x) =
x^3 - 8$', color='black', linewidth=2)
plt.axhline(0, color='black', linewidth=0.8)
plt.axvline(0, color='black', linewidth=0.8)
# Newton-Raphson Iterations setup
x_n = 4.0  # Initial guess
iterations = 6
# Using a colormap to distinguish between many iterati
```

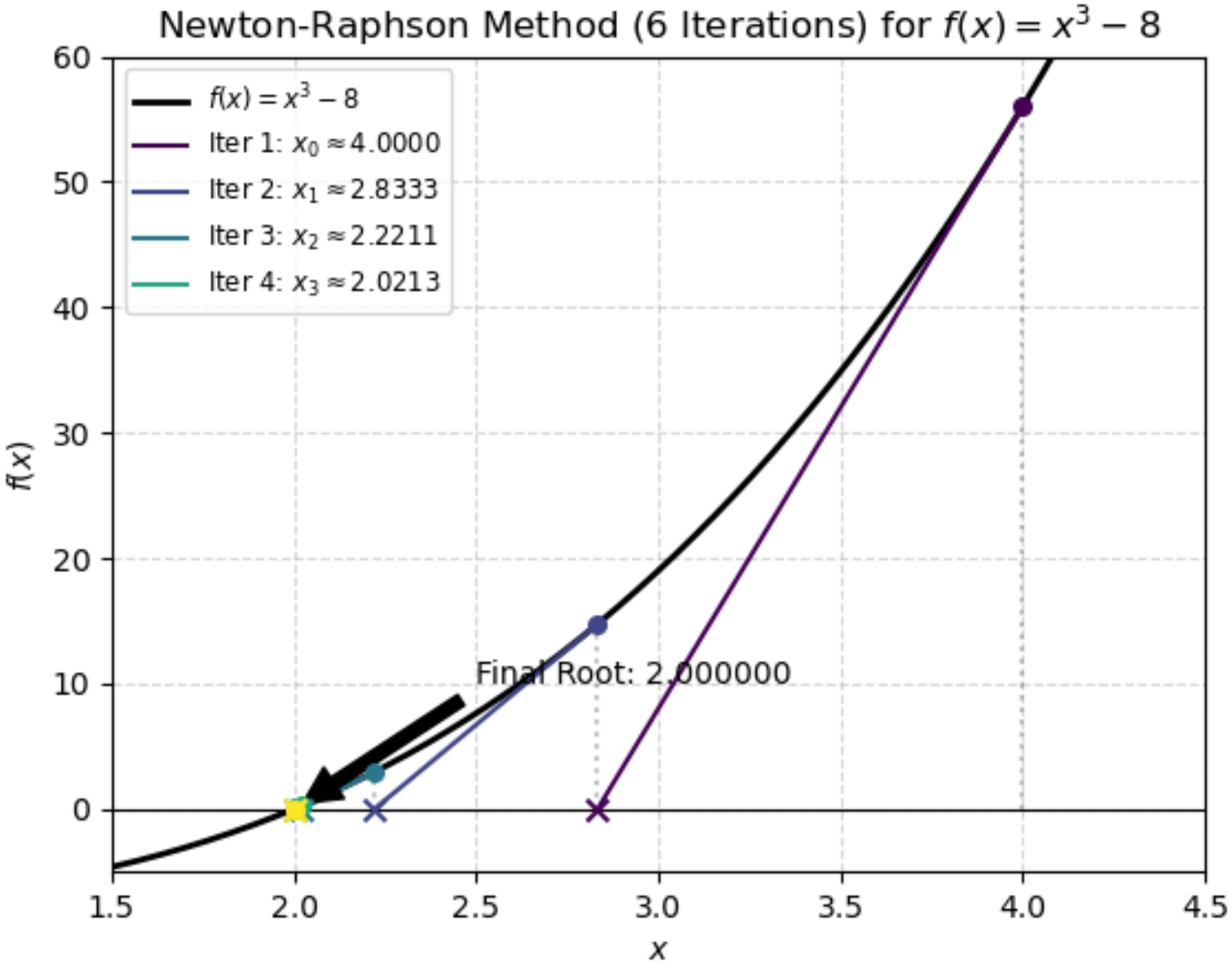
Python Code: Newton-Raphson for Cube Root

```
for i in range(iterations):
y_n = f(x_n)
slope = df(x_n)
# Newton-Raphson step
x_next = x_n - y_n / slope
# Plot vertical segment from (x_n, 0) to (x_n, f(x_n))
plt.vlines(x_n, 0, y_n, colors='gray', linestyles='dot
# Plot the tangent line segment from (x_n, f(x_n)) dow
line_x = np.linspace(x_next, x_n, 100)
line_y = slope * (line_x - x_n) + y_n
# Labeling only the first few to keep the legend clean
label = f'Iter {i+1}: $x_{i} \\approx {x_n:.4f}$' if i
plt.plot(line_x, line_y, color=colors[i], label=label)
```

Python Code: Illustration

```
# Plot the tangent line segment from (x_n, f(x_n)) dow
line_x = np.linspace(x_next, x_n, 100)
line_y = slope * (line_x - x_n) + y_n
# Labeling only the first few to keep the legend clean
label = f'Iter {i+1}: $x_{i} \\approx {x_n:.4f}$' if i
plt.plot(line_x, line_y, color=colors[i], label=label)
# Mark the current point on the curve and the next roo
on the x-axis
plt.scatter([x_n], [y_n], color=colors[i], s=30, zorde
# Update x for next iteration
plt.scatter([x_next], [0], marker='x', color=colors[i]
x_n = x_next
```

Python Code: Illustration

```
# Final root annotation
plt.annotate(f'Final Root: {x_n:.6f}', xy=(x_n, 0),
xytext=(x_n+0.5, 10),
arrowprops=dict(facecolor='black', shrink=0.05),
fontsize=10)
# Formatting the plot
plt.title(r'Newton-Raphson Method
(6 Iterations) for $f(x) = x^3 - 8$')
plt.xlabel('$x$')
plt.ylabel('$f(x)$')
plt.legend(loc='upper left', fontsize='small')
plt.grid(True, linestyle='--', alpha=0.5)
plt.xlim(1.5, 4.5)
plt.ylim(-5, 60)
# Save the plot
plt.show()
```

The Illustration of the Method